



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is impossible to program a mathematically perfect Table-Driven Agent for complex environments like chess because the table would have to contain an action for every possible percept sequence the agent could encounter and as the agents lifetime increases the number of possible percept sequences grows to be enormous making the table too large to store.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The condition `if percept['food'] -> return 'Right'` represents IF food is here, THEN move right, the condition `if percept['wall_ahead'] -> return 'Up'` represents IF there is a wall ahead, THEN move up and `return 'Right'` represents else. Therefore as you can see the agent uses the current percept and selects an action based on an if-then rule without using memory or past experience therefore it implements the Condition-Action rules of the simple reflex agent architecture.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Because the agent percept only contained local information such as `wall_ahead` and not the exact `agent_pos` the agent does not know its exact position in the grid and the agent does not have a percept history to remember what had happened before. Therefore, the agent makes every decision based on only the current percept. If it reaches the same situation again it just sees the same percept and applies the same rule again. Due to partial observability the agent does not get enough information to differentiate different situations and with no percept history the agent cannot remember previous actions and this causes the agent to repeatedly follow the same path or become trapped in a loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

`model_x` and `model_y` are updated by the agent after choosing an action showing the agent's prediction of how the world changes after its action. The agent uses the current percept from the environment to update its location. Therefore Transition model is represented by updating `model_x` and `model_y` and the sensor model is represented by using the current percept(`wall_ahead` and `food_here`) to determine what the agent knows about the current environment.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING